UNIVERSIDAD NACIONAL DE COLOMBIA

Programación Orientada a Objetos 2026-2

Docente: Walter Hugo Arboleda Mazo

Actividad 3 (10% individual)

Ejercicios 8.2 (p. 483) y 8.3 (p. 495) del libro *Ejercicios de Programación Orientada a Objetos con Java y UML* - Leonardo Bermón Angarita

Estudiante: [Escriba aquí su nombre completo]

Repositorio: [Enlace del repositorio de GitHub]

Fecha de entrega: jueves 22 de octubre de 2026

Punto 1 - Ejercicio 8.2 (p. 483): Notas

Enunciado. Programa con interfaz gráfica que solicita cinco notas de un estudiante y muestra el promedio, la desviación estándar, la mayor nota y la menor nota. **Parte propuesta:** (1) mensaje de alerta cuando se ingresan datos no numéricos; (2) es obligatorio ingresar las cinco notas, con alerta si alguna falta.

Enlace del repositorio: [Enlace GitHub: actividad8_2.py]

Código (Python, tkinter)

```
"""
Programación Orientada a Objetos 2026-2 - Actividad 3
Ejercicio 8.2 (p. 483): Notas - componentes gráficos y gestión de eventos.

Se solicitan cinco notas y se calcula el promedio, la desviación estándar,
la nota mayor y la nota menor.

Parte propuesta del ejercicio:
  1. Mensaje de alerta cuando se ingresan datos que no son numéricos.
  2. Es obligatorio ingresar las cinco notas: alerta si alguna falta.
"""
import math
import re
import tkinter as tk
from tkinter import messagebox


class Notas:
    """Modela un arreglo de cinco notas y las operaciones estadísticas sobre ellas."""

    CANTIDAD = 5

    def __init__(self):
        self.lista_notas = [0.0] * Notas.CANTIDAD

    def set_nota(self, indice, valor):
        self.lista_notas[indice] = valor

    def calcular_promedio(self):
        return sum(self.lista_notas) / len(self.lista_notas)

    def calcular_desviacion(self):
        """Desviación estándar poblacional: raíz(suma((x - promedio)^2) / n)."""
        promedio = self.calcular_promedio()
        suma = sum((nota - promedio) ** 2 for nota in self.lista_notas)
        return math.sqrt(suma / len(self.lista_notas))

    def calcular_mayor(self):
        return max(self.lista_notas)

    def calcular_menor(self):
        return min(self.lista_notas)


class VentanaPrincipal(tk.Tk):
    """Ventana con los campos de ingreso de notas, los botones y los resultados."""

    PATRON_NUMERO = re.compile(r"^[+-]?\d+([.,]\d+)?$")

    def __init__(self):
        super().__init__()
        self.title("Notas")
        self.geometry("280x380")
        self.resizable(False, False)
        self.etiquetas_notas = []
        self.campos_notas = []
        self.crear_interfaz()

    def crear_interfaz(self):
        for i in range(Notas.CANTIDAD):
            etiqueta = tk.Label(self, text=f"Nota {i + 1}:", anchor="w")
            etiqueta.place(x=20, y=20 + 30 * i, width=80, height=23)
            campo = tk.Entry(self)
            campo.place(x=105, y=20 + 30 * i, width=135, height=23)
            self.etiquetas_notas.append(etiqueta)
            self.campos_notas.append(campo)

        self.btn_calcular = tk.Button(self, text="Calcular", command=self.calcular)
        self.btn_calcular.place(x=20, y=170, width=100, height=23)
        self.btn_limpiar = tk.Button(self, text="Limpiar", command=self.limpiar)
        self.btn_limpiar.place(x=125, y=170, width=100, height=23)

        self.lbl_promedio = tk.Label(self, text="Promedio = ", anchor="w")
```

```
        self.lbl_promedio.place(x=20, y=210, width=240, height=23)
        self.lbl_desviacion = tk.Label(self, text="Desviación estándar = ", anchor="w")
        self.lbl_desviacion.place(x=20, y=240, width=240, height=23)
        self.lbl_mayor = tk.Label(self, text="Nota mayor = ", anchor="w")
        self.lbl_mayor.place(x=20, y=270, width=240, height=23)
        self.lbl_menor = tk.Label(self, text="Nota menor = ", anchor="w")
        self.lbl_menor.place(x=20, y=300, width=240, height=23)

    def calcular(self):
        """Valida las cinco notas y, si son correctas, muestra los resultados."""
        notas = Notas()
        for i, campo in enumerate(self.campos_notas):
            texto = campo.get().strip()
            # Parte propuesta 2: la nota es obligatoria
            if texto == "":
                self.alerta(f"La Nota {i + 1} no ha sido ingresada.\n"
                            "Es obligatorio ingresar las cinco notas.", campo)
                return
            # Parte propuesta 1: la nota debe ser numérica
            if not self.PATRON_NUMERO.match(texto):
                self.alerta(f"La Nota {i + 1} no es numérica.\n"
                            "Ingrese únicamente números (ej: 3.5).", campo)
                return
            notas.set_nota(i, float(texto.replace(",", ".")))

        self.lbl_promedio.config(text=f"Promedio = {notas.calcular_promedio():.2f}")
        self.lbl_desviacion.config(text=f"Desviación estándar = {notas.calcular_desviacion():.2f}")
        self.lbl_mayor.config(text=f"Nota mayor = {notas.calcular_mayor():.2f}")
        self.lbl_menor.config(text=f"Nota menor = {notas.calcular_menor():.2f}")

    def limpiar(self):
        """Deja en blanco los campos de notas y los resultados."""
        for campo in self.campos_notas:
            campo.delete(0, tk.END)
        self.lbl_promedio.config(text="Promedio = ")
        self.lbl_desviacion.config(text="Desviación estándar = ")
        self.lbl_mayor.config(text="Nota mayor = ")
        self.lbl_menor.config(text="Nota menor = ")
        self.campos_notas[0].focus_set()

    def alerta(self, mensaje, campo):
        messagebox.showwarning("Alerta", mensaje, parent=self)
        campo.focus_set()
        campo.select_range(0, tk.END)


if __name__ == "__main__":
    app = VentanaPrincipal()
    app.mainloop()
```

Interfaz de usuario - Ejercicio 8.2

Ventana inicial y ventana con los resultados calculados (notas 3.5, 4.2, 5, 2.8 y 1.5).

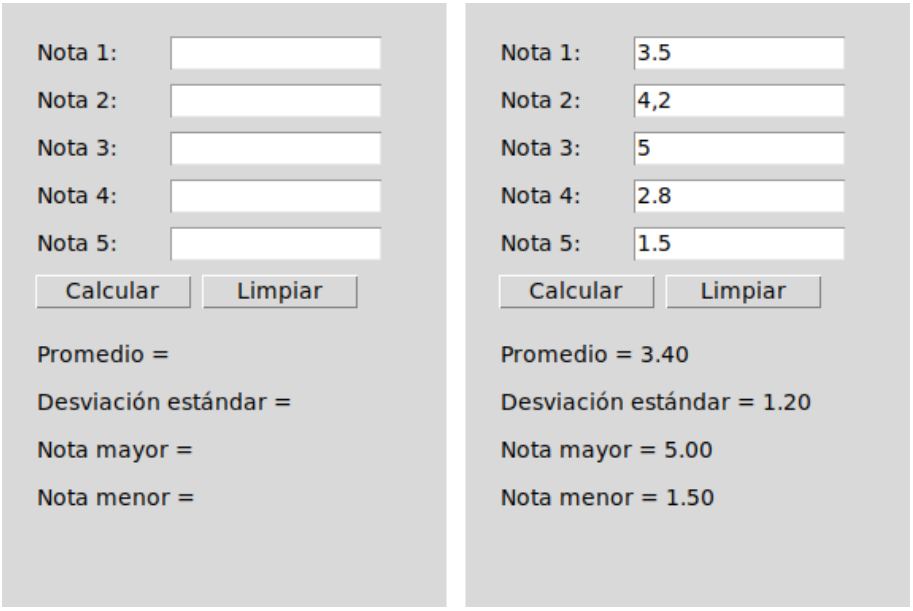


Diagrama de clases - Ejercicio 8.2

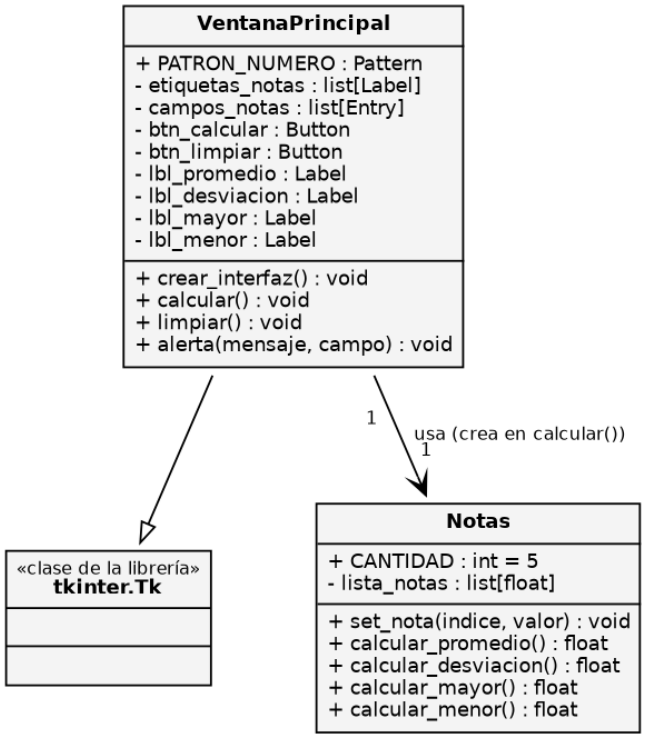
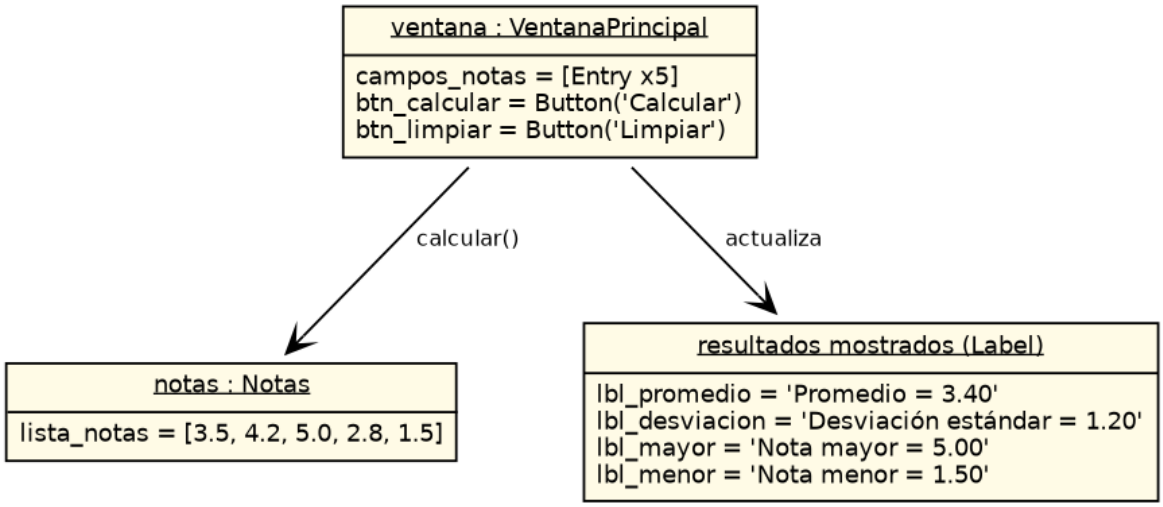
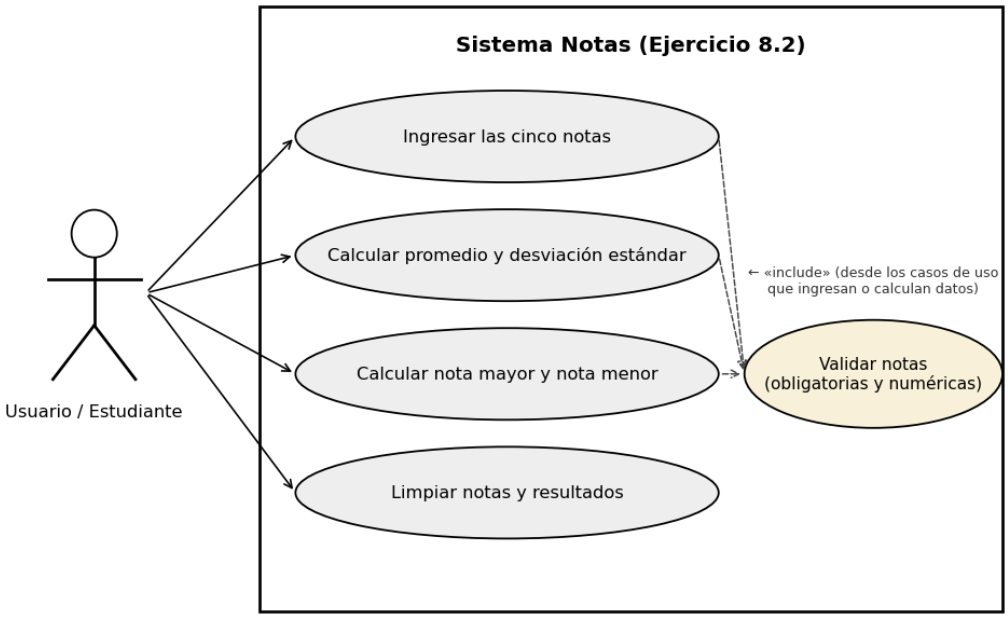


Diagrama de objetos - Ejercicio 8.2



Casos de uso - Ejercicio 8.2



Punto 2 - Ejercicio 8.3 (p. 495): Figuras geométricas

Enunciado. Programa con interfaz gráfica que calcula el volumen y la superficie del cilindro (radio y altura), la esfera (radio) y la pirámide (base, altura y apotema), todo en centímetros, mediante una jerarquía de clases. **Parte propuesta:** (1) agregar el cubo y el prisma; (2) mostrar la imagen correspondiente de cada figura.

Enlace del repositorio: [Enlace GitHub: actividad8_3.py]

Código (Python, tkinter)

```
"""
Programación Orientada a Objetos 2026-2 - Actividad 3
Ejercicio 8.3 (p. 495): Figuras geométricas - gestión de eventos.

Calcula el volumen y la superficie del cilindro, la esfera y la pirámide
mediante una jerarquía de clases.

Parte propuesta del ejercicio:
  1. Agregar el cubo y el prisma (rectangular).
  2. Mostrar la imagen correspondiente en la ventana de cada figura
     (las imágenes se dibujan con tkinter.Canvas, sin archivos externos).
"""
import math
import re
import tkinter as tk
from abc import ABC, abstractmethod
from tkinter import messagebox


# ---------------------------------------------------------------------------
# Jerarquía de clases del modelo
# ---------------------------------------------------------------------------
class FiguraGeometrica(ABC):
    """Clase padre: toda figura tiene un volumen y una superficie."""

    def __init__(self):
        self._volumen = 0.0
        self._superficie = 0.0

    @property
    def volumen(self):
        return self._volumen

    @property
    def superficie(self):
        return self._superficie

    def _calcular(self):
        """Calcula y guarda el volumen y la superficie (se llama en el constructor)."""
        self._volumen = self.calcular_volumen()
        self._superficie = self.calcular_superficie()

    @abstractmethod
    def calcular_volumen(self):
        pass

    @abstractmethod
    def calcular_superficie(self):
        pass


class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
        super().__init__()
        self.radio = radio
        self.altura = altura
        self._calcular()

    def calcular_volumen(self):
        return math.pi * self.radio ** 2 * self.altura

    def calcular_superficie(self):
        return 2 * math.pi * self.radio * self.altura + 2 * math.pi * self.radio ** 2


class Esfera(FiguraGeometrica):
    def __init__(self, radio):
        super().__init__()
        self.radio = radio
        self._calcular()

    def calcular_volumen(self):
        return (4.0 / 3.0) * math.pi * self.radio ** 3
```

```
    def calcular_superficie(self):
        return 4 * math.pi * self.radio ** 2


class Piramide(FiguraGeometrica):
    """Pirámide de base cuadrada; el apotema es la altura de cada cara lateral."""

    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = base
        self.altura = altura
        self.apotema = apotema
        self._calcular()

    def calcular_volumen(self):
        return (self.base ** 2 * self.altura) / 3.0

    def calcular_superficie(self):
        area_base = self.base ** 2
        area_laterales = 4 * (self.base * self.apotema / 2.0)  # 4 triángulos
        return area_base + area_laterales


class Cubo(FiguraGeometrica):
    """Parte propuesta."""

    def __init__(self, lado):
        super().__init__()
        self.lado = lado
        self._calcular()

    def calcular_volumen(self):
        return self.lado ** 3

    def calcular_superficie(self):
        return 6 * self.lado ** 2


class Prisma(FiguraGeometrica):
    """Prisma rectangular (ortoedro). Parte propuesta."""

    def __init__(self, largo, ancho, alto):
        super().__init__()
        self.largo = largo
        self.ancho = ancho
        self.alto = alto
        self._calcular()

    def calcular_volumen(self):
        return self.largo * self.ancho * self.alto

    def calcular_superficie(self):
        return 2 * (self.largo * self.ancho + self.largo * self.alto + self.ancho * self.alto)


# ------------------------------------------------------------------------------
# Interfaz gráfica
# ------------------------------------------------------------------------------
class PanelImagen(tk.Canvas):
    """Canvas que dibuja la imagen de la figura geométrica."""

    RELLENO = "#78AAE1"
    CLARO = "#AACDF0"
    OSCURO = "#5082BE"
    BORDE = "#1E3C6E"

    def __init__(self, parent, tipo, ancho=340, alto=120):
        super().__init__(parent, width=ancho, height=alto, bg="white", highlightthickness=0)
        cx, cy = ancho // 2, alto // 2
        getattr(self, f"_dibujar_{tipo}")(cx, cy)

    def _dibujar_cilindro(self, cx, cy):
        rx, ry, alto = 40, 12, 60
        top = cy - alto // 2
        self.create_oval(cx - rx, top + alto - ry, cx + rx, top + alto + ry,
                         fill=self.RELLENO, outline=self.BORDE, width=2)
        self.create_rectangle(cx - rx, top, cx + rx, top + alto, fill=self.RELLENO, outline="")
        self.create_arc(cx - rx, top + alto - ry, cx + rx, top + alto + ry, start=180, extent=180,
                        style="arc", outline=self.BORDE, width=2)
        self.create_line(cx - rx, top, cx - rx, top + alto, fill=self.BORDE, width=2)
        self.create_line(cx + rx, top, cx + rx, top + alto, fill=self.BORDE, width=2)
        self.create_oval(cx - rx, top - ry, cx + rx, top + ry,
                         fill=self.CLARO, outline=self.BORDE, width=2)

    def _dibujar_esfera(self, cx, cy):
        r = 45
        self.create_oval(cx - r, cy - r, cx + r, cy + r, fill=self.RELLENO, outline=self.BORDE, width=2)
        self.create_oval(cx - 30, cy - 33, cx - 4, cy - 12, fill=self.CLARO, outline="")
        self.create_arc(cx - r, cy - 12, cx + r, cy + 12, start=180, extent=180, style="arc",
                        outline=self.BORDE, dash=(4, 3))
    def _dibujar_piramide(self, cx, cy):
```

```
        apice = (cx, cy - 45)
        izq, frente, der = (cx - 50, cy + 30), (cx + 5, cy + 45), (cx + 55, cy + 25)
        self.create_polygon(*apice, *izq, *frente, fill=self.CLARO, outline=self.BORDE, width=2)
        self.create_polygon(*apice, *frente, *der, fill=self.OSCURO, outline=self.BORDE, width=2)
        oculto = (cx, cy + 10)
        for destino in (izq, der, apice):
            self.create_line(*oculto, *destino, fill=self.BORDE, dash=(4, 3))

    def _dibujar_caja(self, cx, cy, w, h, d):
        ox, oy = d, -int(d * 0.6)
        x = cx - (w + ox) // 2
        y = cy - (h - oy) // 2 - oy
        self.create_polygon(x, y, x + ox, y + oy, x + w + ox, y + oy, x + w, y,
                            fill=self.CLARO, outline=self.BORDE, width=2)
        self.create_polygon(x + w, y, x + w + ox, y + oy, x + w + ox, y + h + oy, x + w, y + h,
                            fill=self.OSCURO, outline=self.BORDE, width=2)
        self.create_rectangle(x, y, x + w, y + h, fill=self.RELLENO, outline=self.BORDE, width=2)

    def _dibujar_cubo(self, cx, cy):
        self._dibujar_caja(cx, cy, 50, 50, 30)

    def _dibujar_prisma(self, cx, cy):
        self._dibujar_caja(cx, cy, 85, 40, 40)


class VentanaFigura(tk.Toplevel, ABC):
    """
    Ventana base para calcular el volumen y la superficie de una figura.
    Cada subclase define sus campos y cómo crear su figura.
    """

    ANCHO = 340
    ALTO_IMAGEN = 120
    PATRON_NUMERO = re.compile(r"^\d+([.,]\d+)?$")

    def __init__(self, parent, titulo, tipo, nombres_campos):
        tk.Toplevel.__init__(self, parent)
        self.title(titulo)
        self.resizable(False, False)
        self.campos = []
        self.nombres = nombres_campos

        PanelImagen(self, tipo).place(x=0, y=0, width=self.ANCHO, height=self.ALTO_IMAGEN)

        y = self.ALTO_IMAGEN + 15
        for nombre in nombres_campos:
            tk.Label(self, text=nombre, anchor="w").place(x=20, y=y, width=115, height=23)
            campo = tk.Entry(self)
            campo.place(x=140, y=y, width=135, height=23)
            self.campos.append(campo)
            y += 30

        tk.Button(self, text="Calcular", command=self.calcular).place(x=20, y=y + 5, width=100, height=23)
        self.lbl_volumen = tk.Label(self, text="Volumen (cm³) = ", anchor="w")
        self.lbl_volumen.place(x=20, y=y + 40, width=300, height=23)
        self.lbl_superficie = tk.Label(self, text="Superficie (cm²) = ", anchor="w")
        self.lbl_superficie.place(x=20, y=y + 70, width=300, height=23)
        self.geometry(f"{self.ANCHO}x{y + 110}")

    @abstractmethod
    def crear_figura(self, valores):
        """Crea la figura concreta con los valores ingresados (en orden de los campos)."""

    def calcular(self):
        valores = []
        for nombre, campo in zip(self.nombres, self.campos):
            etiqueta = nombre.replace(":", "")
            texto = campo.get().strip()
            if texto == "":
                return self.alerta(f"Debe ingresar el dato: {etiqueta}", campo)
            if not self.PATRON_NUMERO.match(texto):
                return self.alerta(f'El dato "{etiqueta}" debe ser un número positivo (ej: 4.5).', campo)
            valor = float(texto.replace(",", "."))
            if valor <= 0:
                return self.alerta(f'El dato "{etiqueta}" debe ser mayor que cero.', campo)
            valores.append(valor)
        figura = self.crear_figura(valores)
        self.lbl_volumen.config(text=f"Volumen (cm³) = {figura.volumen:.2f}")
        self.lbl_superficie.config(text=f"Superficie (cm²) = {figura.superficie:.2f}")

    def alerta(self, mensaje, campo):
        messagebox.showwarning("Alerta", mensaje, parent=self)
        campo.focus_set()
        campo.select_range(0, tk.END)


class VentanaCilindro(VentanaFigura):
    def __init__(self, parent):
        super().__init__(parent, "Cilindro", "cilindro", ["Radio (cms):", "Altura (cms):"])

    def crear_figura(self, v):
```

```
        return Cilindro(v[0], v[1])


class VentanaEsfera(VentanaFigura):
    def __init__(self, parent):
        super().__init__(parent, "Esfera", "esfera", ["Radio (cms):"])

    def crear_figura(self, v):
        return Esfera(v[0])


class VentanaPiramide(VentanaFigura):
    def __init__(self, parent):
        super().__init__(parent, "Pirámide", "piramide",
                         ["Base (cms):", "Altura (cms):", "Apotema (cms):"])

    def crear_figura(self, v):
        return Piramide(v[0], v[1], v[2])


class VentanaCubo(VentanaFigura):
    def __init__(self, parent):
        super().__init__(parent, "Cubo", "cubo", ["Lado (cms):"])

    def crear_figura(self, v):
        return Cubo(v[0])


class VentanaPrisma(VentanaFigura):
    def __init__(self, parent):
        super().__init__(parent, "Prisma", "prisma", ["Largo (cms):", "Ancho (cms):", "Alto (cms):"])

    def crear_figura(self, v):
        return Prisma(v[0], v[1], v[2])

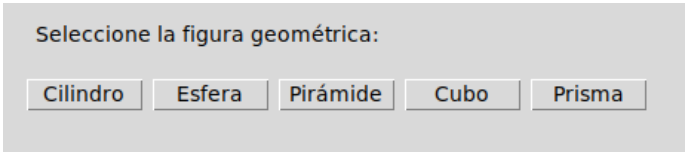

class VentanaPrincipal(tk.Tk):
    """Ventana principal: un botón por figura que abre su ventana de cálculo."""

    def __init__(self):
        super().__init__()
        self.title("Figuras")
        self.geometry("460x100")
        self.resizable(False, False)
        tk.Label(self, text="Seleccione la figura geométrica:", anchor="w").place(x=20, y=10, width=300, height=23)
        figuras = [("Cilindro", VentanaCilindro), ("Esfera", VentanaEsfera),
                   ("Pirámide", VentanaPiramide), ("Cubo", VentanaCubo), ("Prisma", VentanaPrisma)]
        self.botones = {}
        for i, (texto, clase) in enumerate(figuras):
            boton = tk.Button(self, text=texto, command=lambda c=clase: c(self))
            boton.place(x=15 + 85 * i, y=50, width=80, height=23)
            self.botones[texto] = boton

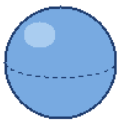

if __name__ == "__main__":
    app = VentanaPrincipal()
    app.mainloop()
```

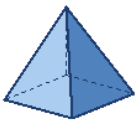
Interfaz de usuario - Ejercicio 8.3

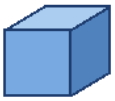
Ventana principal y ventanas de cada figura con un cálculo de ejemplo.











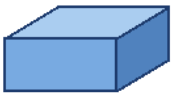


Diagrama de clases - Ejercicio 8.3

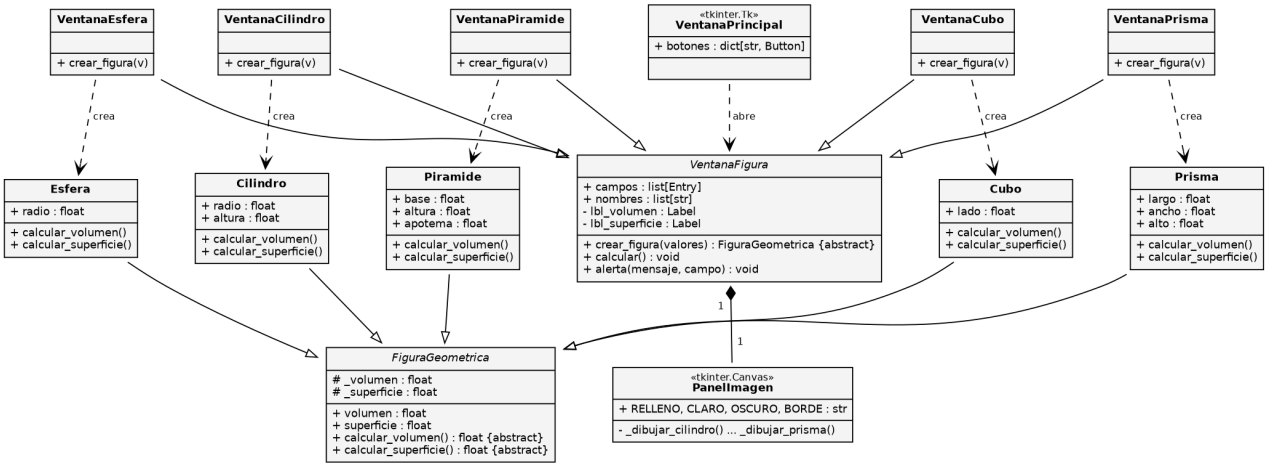
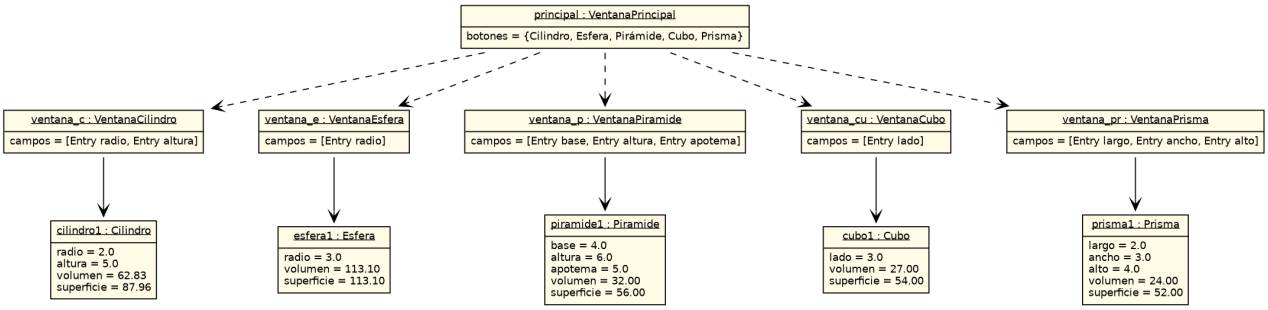


Diagrama de objetos - Ejercicio 8.3



Casos de uso - Ejercicio 8.3

